DATA/CS 172: Python for Data Analysis

Intro to NumPy Arrays

Prof. Travis Mandel

10/30/2024

UNIVERSITY of HAWAI'I® HILO

Reminders/Announcements

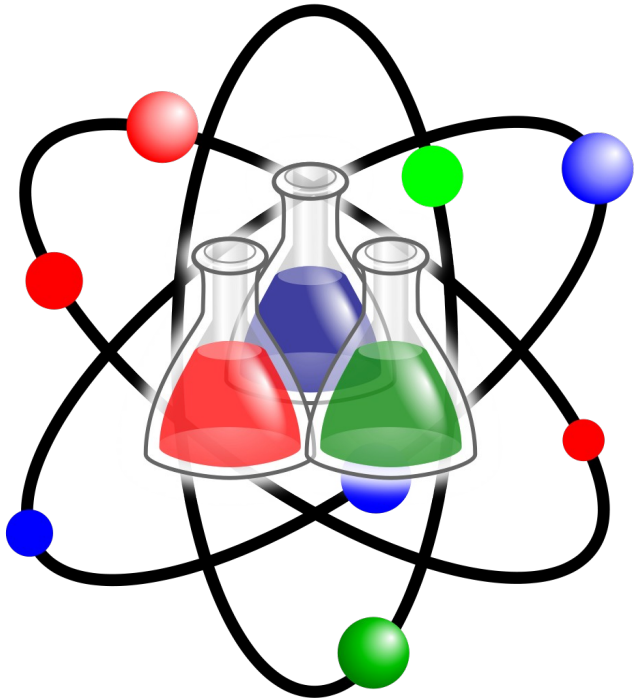
- Lab 15 final deadline today
- Lab 16 due today

What are numpy arrays?

- We already know how to do a list in Python…
- Many Python programs use a similar concept, called a numpy array…
  - Example:
    - How do we get a list of numbers from 0 to 99 in Python?
      - list(range(100))
    - How do we get a numpy array of numbers from 0 to 99 in Python?
      - import numpy as np
      - np.arange(100)
      - Should have this installed as it is installed with matplotlib
- One big difference: cannot append!
  - Once a numpy array is created its length does not change

Ok, but why??

- Reason 1: Many scientific computing libraries take numpy arrays as input, and/or produce them as output
  - Machine learning libraries too!



Reason 2: Convenience

- Don’t need for loops to do simple (vector) operations
- E.g. want to multiply every element of a list by 10
  - Demo
- Useful built-in functions
  - arr.sum()
    - Takes the sum of the array elements
  - arr.mean()
    - Takes the mean of the array elements
  - arr.std()
    - Takes the standard deviation of the array elements

Reason 3: Speed

- Numpy is actually a wrapper around a C implementation
  - C code is much faster because it does not have to do all the complicated checking, works directly on memory, etc.

Creating numpy array from existing list

- So far, the only way we can create is with np.arange()
- We know python lists are extremely similar to numpy arrays
- What if you want to convert a list to a numpy array?
  - np.array([3,5,7]) → [3 5 7]

Can do most things you would on a normal list

- Index
  - myArr[0] #gets first element
- Take the length
  - len(myArr) # gets number of elements
- New: Can do “all-at-once” operations
  - myArr + 5 #Adds 5 to EVERY element
- But **cannot** append!
  - myArr.append(3) # causes an error

Creating an all-zero array

- np.zeros(aLength)
  - [0. 0. 0. 0. ….]

Working with two numpy arrays…

- If two numpy arrays joined by arithmetic operator +,*, etc.
  - Operation proceeds element-wise
- Each element matched with other element, then operation applied
  - Will cause errors if different lengths
- [0 1 2 3 4] + [1 2 1 2 1] = [1 3 3 5 5]

Element-wise operations

- When you do standard operations (+,*, -, etc.) on an array and a number, it applies that to ALL elements

- [0 1 2 3 4] * 2 = [0 2 4 6 8]
- [0 1 2 3 4] -1 = [-1 0 1 2 3]
- Works on +=, -=, etc. too!
- This is fast
  - Demo

Changing types

- Common problem: When reading data from a file, it comes in as the wrong type
  - Example: comes in as strings instead of ints
- Currently, if we have a list of strings, we need a for loop to convert it to a list of ints
  - Seems kind of overkill
- Numpy makes this very easy
  - stringList = np.array([“1”, “5”, “3”]
  - intList = stringList.astype(int) ← Now an array of ints!
  - Also works for other types (e.g. float)
- Arr.dtype shows the type of an array
  - More specific than Python types
    - Float64 can represent more values/digits than float32
    - Uint8 can represent less values compared to int64
      - Only 0-255

Simple lab out

- Getting started in numpy
- More complicated one later